

[< Основы алгоритмов](#)

6.2 Жадные алгоритмы

Авторы



Алексей Толстиков



Александр Куликов

В этом параграфе вы разберётесь с жадными алгоритмами — стратегиями, которые на каждом шаге выбирают «самое выгодное» решение в надежде на общий успех. Мы увидим, когда такая тактика действительно работает, а когда — приводит к ошибкам. На примере задачи о бронировании переговоров вы научитесь оценивать и доказывать корректность жадных решений.

Ключевые вопросы параграфа

- Что такое жадный алгоритм и в чём его идея?
- Почему не всякая жадная стратегия приводит к оптимальному решению?
- Как формализовать и доказать корректность жадного алгоритма?

Итерационный принцип алгоритмов

Многие алгоритмы — это итерационные процедуры: с каждым повтором они делают выбор из определенного количества вариантов.

Например, для кассира задача «Размен» может быть представлена как

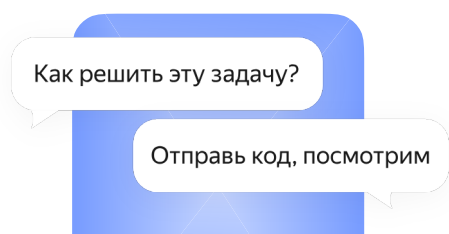
последовательность решений: какую монету (из d ценностей) вернуть первой, какую второй и так далее. Некоторые из этих вариантов приведут к правильному ответу, а некоторые — нет.

Принцип работы жадного алгоритма

При каждом повторе жадный алгоритм выбирает «самый привлекательный» вариант. Например, самый большой номинал из доступных монет. В случае с американскими деньгами `Change` использует номиналы четвертак (25 центов), дайм (10 центов), никель (5 центов) и пенни (1 цент), чтобы выдать сдачу, в данном порядке. Разумеется, мы показывали, как такой «жадный» подход приводит к неправильным результатам при добавлении монет некоторых новых номиналов.

Пример с телефоном

В примере с телефоном «жадная» стратегия состояла бы в том, чтобы идти на звук, пока вы его не найдете. Но есть проблема: между вами и телефоном может оказаться стена (или хрупкая ваза). К сожалению, такие сложности часто возникают и в реальных задачах.



Вступайте в сообщество хендбука

Здесь можно найти единомышленников, экспертов и просто интересных собеседников. А ещё — получить помощь или поделиться знаниями.

Вступить

Во многих случаях «жадный» подход выглядит естественным и очевидным, но может оказаться неправильным.

Задача «Бронирование переговоров»

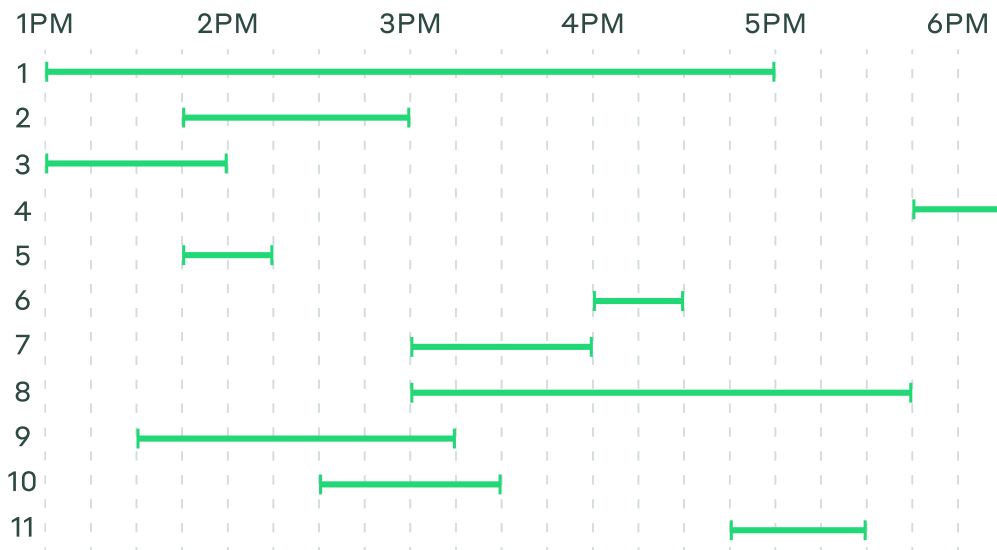
В задаче «Бронирование переговоров» вам дается несколько временных отрезков, и нужно выбрать как можно больше отрезков таким образом, чтобы ни один из них не пересекался с другим (отрезки пересекаются, если у них есть общая точка).

Название задачи основано на следующей гипотетической ситуации. Представьте, что у вас есть зал для переговоров, и вам присылают заявки на бронирование 11

компаний.

1	01:00PM—05:00PM
2	01:45PM—03:00PM
3	01:00PM—02:00PM
4	05:45PM—06:15PM
5	01:45PM—02:15PM
6	04:00PM—04:30PM
7	03:00PM—04:00PM
8	03:00PM—05:45PM
9	01:30PM—03:15PM
10	02:30PM—03:30PM
11	04:45PM—05:30PM

Нельзя удовлетворить все запросы (так как некоторые из них пересекаются), но мы хотим удовлетворить как можно больше. Для этого мы представим входные данные более удобным способом.

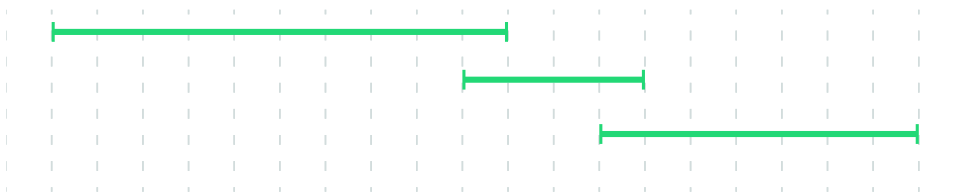


Так как мы говорим о «жадных» стратегиях, давайте поэкспериментируем с разными «наиболее выгодными» подходами. Интуиция может нам подсказать, что нужно выбрать самый короткий отрезок, удалить пересекающиеся отрезки и повторить данное действие.

Остановитесь и подумайте:

Всегда ли это приведет к оптимальному решению?

► **Ответ**

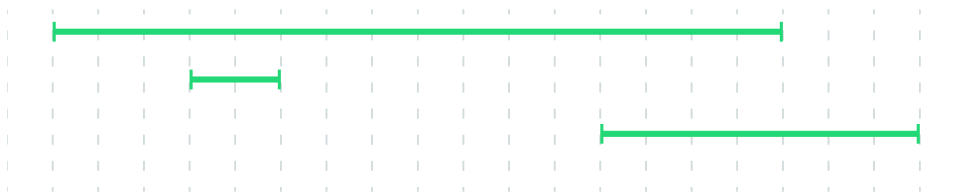


Возможно, логичнее было бы выбрать отрезок слева (тот, что начинается раньше всех), убрать все остальные и повторить данное действие.

Остановитесь и подумайте:

Всегда ли это приведет к оптимальному решению?

► **Ответ**



Остановитесь и подумайте:

Может, есть и другие «жадные» подходы?

► **Ответ**

Оказывается, что следующий «жадный» алгоритм максимизирует количество непересекающихся отрезков: выбрать чемпионский отрезок, убрать все пересекающиеся с ним отрезки, повторить выбор.

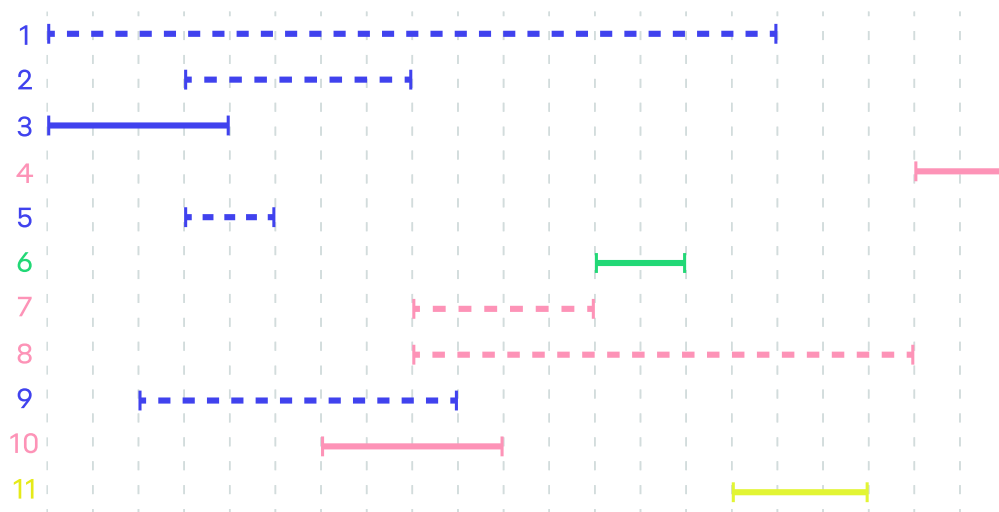
Упражнение

Докажите, что если набор непересекающихся отрезков не содержит чемпионский отрезок, то при замене первого отрезка в этом наборе на чемпионский мы получаем набор непересекающихся отрезков.

Вот мы и нашли оптимальную «жадную» стратегию. И действительно, если существует решение задачи, включающее в себя чемпионский интервал, мы можем выбрать этот интервал на первом шаге и решить задачу выбора непересекающихся отрезков из оставшихся.

В нашем примере алгоритм работает следующим образом.

- Выбрать сегмент 3 и отбросить сегменты 1, 2, 5, и 9.
- Выбрать сегмент 10 и отбросить сегменты 7 и 8.
- Выбрать сегмент 6.
- Выбрать сегмент 11.
- Выбрать сегмент 4.



Что дальше

Теперь вы понимаете, как устроены жадные алгоритмы и почему они могут быть одновременно простыми и опасными. Вы научились проверять, работает ли жадная стратегия в конкретной задаче, и даже доказывать её оптимальность.

Далее — динамическое программирование. Это подход, который позволяет решать сложные задачи, разбивая их на подзадачи и используя уже найденные решения. Вы узнаете, как избежать повторных вычислений и добиться эффективности там, где перебор и жадность не справляются.

А пока вы не ушли дальше — закрепите материал на практике:

- Отметьте, что урок прочитан, при помощи кнопки ниже.
- Пройдите мини-квиз, чтобы проверить, насколько хорошо вы усвоили тему.
- Перейдите к [задачам](#) этого параграфа и потренируйтесь.
- Перед этим — загляните в короткий [гайд](#) о том, как работает система проверки.

Хотите обсудить, задать вопрос или не понимаете, почему код не работает? Мы всё предусмотрели — вступайте в [сообщество Хендбука](#)! Там студенты помогают друг другу разобраться.

Ключевые выводы параграфа

- Жадный алгоритм на каждом шаге выбирает наиболее выгодный вариант, не заглядывая вперёд. Такой подход прост и работает быстро, но не всегда даёт оптимальное решение.

- Чтобы жадная стратегия была корректной, необходимо доказать, что локальный выбор всегда ведёт к глобально лучшему результату.
- В некоторых задачах — например, при выборе максимального числа непересекающихся отрезков — удаётся найти и обосновать корректную жадную стратегию.

Параграф прочитан

Выполните задачи урока

0 / 5 выполнено

Выполнять задачи



Пройдите квиз по параграфу

Чтобы закрепить пройденный материал

Начать

 Сообщить об ошибке

Предыдущий параграф

6.1. Полный перебор и оптимизация перебора

Метод полного перебора позволяет проанализировать все возможные исходы поставленной задачи и выбрать самый оптимальный. Перечисление возможных вариантов само по себе может оказаться нетривиальным и увлекательным.



Следующий параграф

6.3. Динамическое программирование

Здесь мы найдём оптимальную стратегию действий в игре «Камни». Решив большее количество меньших экземпляров задачи, мы сможем гарантированно определить, какой из игроков непременно победит.



Яндекс Образование

Яндекс Учебник	Исследования	О нас
Яндекс Лицей	Хендбуки	Обратная связь
Яндекс Практикум	Карты IT-навыков	Пользовательское соглашение
Школа анализа данных	База знаний	Сайт образовательной организации
Программы в университетах	Журнал	Сведения об образовательной организации
	События	

Рассылка Бот   

Образовательные услуги оказываются АНО ДПО «Образовательные технологии Яндекса» на основании [Лицензии № Л035-01298-77/00185314](#) от 24 марта 2015 года.
© 2026 Яндекс, АНО ДПО «Образовательные технологии Яндекса».